

The BRITISH UNIVERSITY IN EGYPT

# 24CSAI02I

# Visual Computing and Image Processing

Amr Mohamed Galal Eldin Ali Rizk

235355

Artificial Intelligence

(A4)

Submitted to:

*Prof. Ahmed Fahmy*

2025

## Table of Contents

**1.Project Overview…….3**

**2.Project Concept ……..….3**

**3.Project Implementation …….3**

**4. Final Output and Observations:……..……7**

**6.Project Challenges ……..……8**

**7.Conclusion ……..……8**

**8.Images ……8**

# Cartoonize The Image

## Project Overview

Project goal entails building an automated facial detection system leveraging machine learning that subsequently transfigures detected faces into cartoonish renditions rapidly. Object detection and image processing techniques are combined in this process leveraging a trained YOLOv8 model and various OpenCV filters effectively. Overall idea remains pretty simple yet wickedly powerful given any input image system should identify human face and subsequently apply filters transforming it into cartoonized version rapidly. I tackled project with focus on practicality and theoretical underpinnings simultaneously from start amidst complexity and hazy objectives somehow. Project lets me explore creative side of computer vision pretty deeply through various image effects and somewhat intricate transformations.

## Project Concept

A real-world image gets stylized rather grotesquely into somewhat cartoonish sketchy visuals through cartoonization which involves a transformation technique. Smoothing and color simplification often occur alongside edge detection generally in a fairly complicated process. Typical cartoon images often boast stark vivid edges and curiously minimal color gradations mimicking appearance of hand drawn illustration quite effectively. Cartoonization in this project focuses heavily on human faces. We need locating a face within an image first pretty accurately. YOLO swoops in right there somehow. YOLO is a rather sophisticated deep learning model ostensibly devised for swift object detection in real time with remarkable accuracy. I trained custom YOLOv8 model painstakingly using labeled dataset and applied various image processing operations subsequently cartoonizing detected facial regions.

## Project Implementation

**1.** first step is installing ultralytics as YOLOv8 requires this package, which allows us to train and use YOLO models directly in Python with minimal setup.

```
!pip install ultralytics
```

**2.** second step involved importing necessary libraries:

- cv2 is used for all image processing tasks.
- Numpy is used for some mathematical operations.
- ultralytics provides the implementation for YOLOv8, a state-of-the-art object detection model and torch is used to generate the YOLO model
- matplotlib.pyplot is used for displaying images during development.

```
import torch
import cv2
import numpy as np
import matplotlib.pyplot as plt
from ultralytics import YOLO
```

**3.** third step I Trained the YOLO Model

```
model = YOLO("yolov8n.pt")
path = r"C:\\Users\\Dell\\Desktop\\Project\\Project\\data.yaml"
model.train(data=path, epochs=20, imgsz=128)
```

Here, I initialized the YOLO model with the "yolov8n.pt" model, which are faster and efficient for smaller projects. I trained it using a custom dataset I downloaded from Roboflow Universe website , the structure of which is defined in the data.yaml file present in the downloaded dataset in which I inserted it as the path. The training was done over 20 epochs using an image size of 128x128 for better experimentation. The dataset was especially for face detection in which it carefully prepared to include images of faces labeled correctly, so that the model learns to recognize the facial structure across various orientations and lighting conditions and across any image that I will provide it.

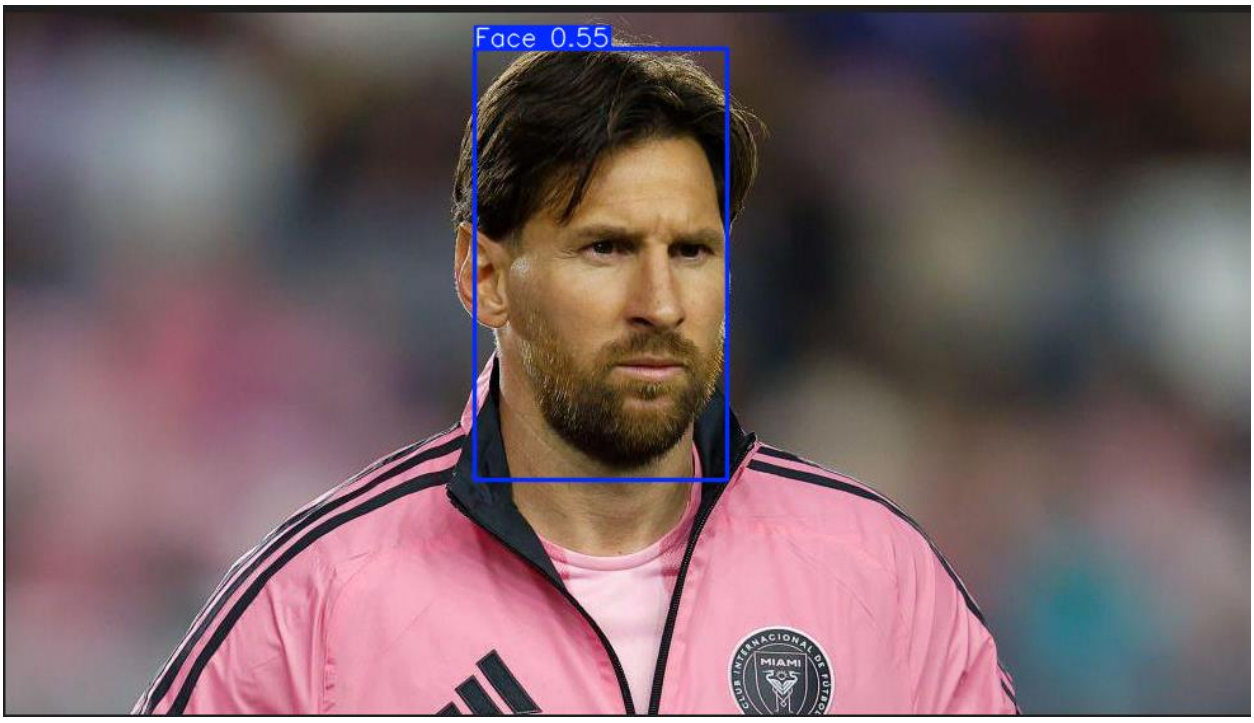
**4.** model = YOLO("runs\\detect\\train7\\weights\\best.pt")

results = model(["messi.jpg"], iou=0)

After training, I loaded the best-performing model by best.pt which I got it from training the model , I preferred not to use last.pt because not every last iteration could be the best iteration and then I used it to run inference on an image I downloaded from google for Messi (messi.jpg), and  I made the iou( Intersection Over Union)=0 inorder not to overlap if there more than 1 face detected.The model outputs the bounding boxes for detected faces in the image.

results[0].show()

This line visually displays the detected face(s) with bounding boxes overlaid on the image.



**5.**

```
for result in results:

for box in result. boxes.xyxy:

x1, y1, x2, y2 = map(int, box[:4])

image = cv2.imread("messi.jpg")

face_crop = image[y1:y2, x1:x2]

cv2.imshow("cropped face",face_crop)

cv2.waitKey(0)
```

In this step, I looped through the detection results, extracted the coordinates of the bounding box, and then cropped the face from the original image using standard slicing. This isolates the facial region which we then will cartoonize. Also, I made sure to read the image and reference the correct slice using the coordinates from YOLO output. And then I displayed the cropped image.

6. For cartoonizing the cropped face It involved several steps:

Convert to Grayscale: This step convert the image to a black and white image, it simplifies the image and makes edge detection more effective.

```
gray = cv2.cvtColor(face_crop, cv2.COLOR_BGR2GRAY)
```

Apply Median Blur: Median blur helps reduce noise while preserving edges, which is crucial for displaying cartoon-style lines.

```
gray = cv2.medianBlur(gray, 5)
```

Edge Mask using Adaptive Thresholding**:** This creates a binary mask highlighting the major edges, forming the "cartoon outline."

```
masks = cv2.adaptiveThreshold(gray, 255, cv2.ADAPTIVE_THRESH_MEAN_C,
cv2.THRESH_BINARY, 11, 3)
```

Color Smoothing with Bilateral Filter: Bilateral filtering smooths colors while keeping edges sharp. It gives the image painting-like effect.

```
smoothed = cv2.bilateralFilter(face_crop, 9, 250, 250)
```

Final Cartoon Image: The cartoon is created by applying the edge mask to the smoothed color image using the and bitwise to combine them

```
cartoon = cv2.bitwise_and(smoothed, smoothed, mask=masks)
```

## Final Output and Observations:

The final output is a stylized cartoon face gets extracted pretty cleverly from real images occasionally with fantastic results naturally. Model performance was remarkably robust on frontal faces under fairly general lighting. Cartoon effect looked pretty darn realistic capturing main facial features with simplified colors and bold outlines quite consistently with most expectations. Face detection was finely tuned to my dataset by training the YOLO model with.



## Project Challenges:

• Preparing a good dataset and labeling it accurately took more time than expected but was crucial for model accuracy.

• Training YOLO model required experimentation with batch sizes, learning rates, and image sizes to balance speed and performance.

• I explored various filters before settling on bilateral filtering and adaptive thresholding for best cartoon effect.

## Conclusion:

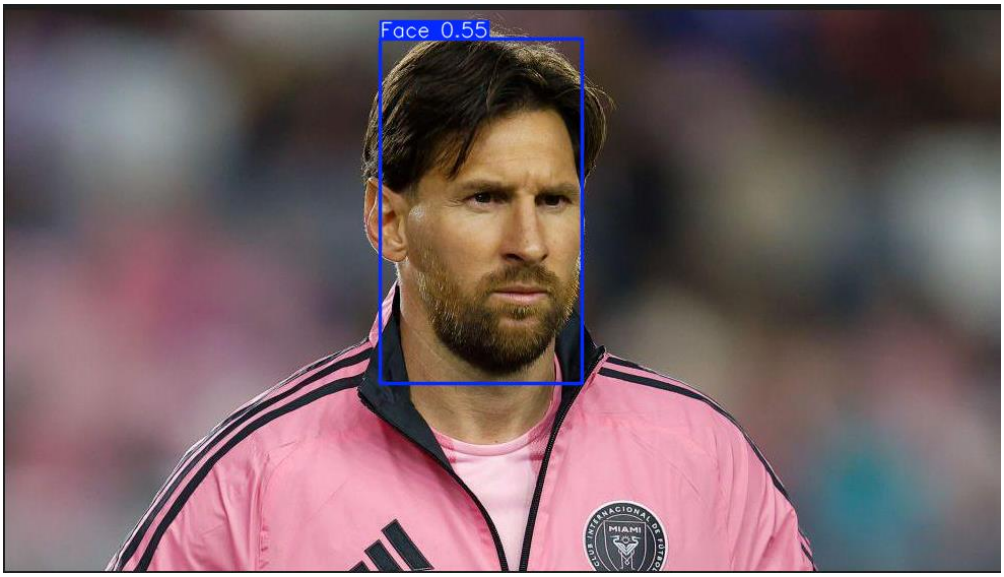
Machine learning merged rather satisfyingly with creative image processing techniques in a deeply fulfilling project undertaken recently with great enthusiasm. It increased my grasp of YOLO object detection and imparted knowledge on getting ML outputs with traditional image manipulation techniques via Ultralytics and OpenCV pretty effectively. Pipeline modularity lets me plug in various effects or detection models easily in future works with relative simplicity. This work embodies a quintessential visual computing application both technically and practically and it provided me with passion for this field.

## The Images:

The Original Image:



The face detection using trained YOLO model:




The cropped face after detection:




The final Cartoonized image:

